

Горчак Дмитрий, 212. Кластерный анализ облака точек: матрица расстояний, порог, связные компоненты

Кластер (cluster – сгусток) – группа точек, которые лежат близко друг к другу и заметно дальше от остальных. Задача кластерного анализа: по одним только координатам точек, без всяких подсказок, определить, сколько таких групп в облаке и какая точка какой группе принадлежит.

Ниже разобрана цепочка, которой эта задача решается: матрица попарных расстояний, выбор порога близости, двоичная матрица связей, поиск связных компонент двумя обходами – волновым и с прорубанием просеки, – подбор порога по минимальному покрывающему дереву и ускорение расчёта сеткой блоков. Примеры взяты из учебного проекта `cones`: вокруг каждого конуса рассыпано по два десятка точек с гауссовым разбросом, а программа должна восстановить сгустки, ничего не зная о конусах.

Матрица расстояний

Первый шаг – сосчитать расстояние между каждой парой точек и сложить результаты в квадратную таблицу n на n , где n – число точек. Элемент в строке i и столбце j равен расстоянию от точки i до точки j . Такая таблица называется матрицей расстояний.

У неё два очевидных свойства: на главной диагонали стоят нули (расстояние от точки до самой себя), и она симметрична – расстояние от i до j равно расстоянию от j до i . Поэтому считать нужно только верхнюю половину, а нижнюю заполнить копированием; это сразу вдвое сокращает работу.

```
for (std::size_t i = 0; i < n; ++i) {
    for (std::size_t j = i + 1; j < n; ++j) {
        float distance = points[i].distance_to(points[j]);

        distances[i][j] = distance;
        distances[j][i] = distance;    /* матрица симметрична */
    }
}
```

Цена такой матрицы – квадрат числа точек: и по времени, и по памяти. Для 240 точек это 57 600 чисел, что совершенно не страшно, но для ста тысяч точек матрица уже не помещается в память. К этому мы вернёмся в разделе о сетке блоков.

Порог и двоичная матрица связей

Само расстояние ещё ничего не говорит о группах. Нужно решить, какое расстояние считать «близко». Это число называется порогом. Задав его, матрицу расстояний превращают в двоичную матрицу связей: на месте каждого расстояния ставится единица, если оно не больше порога, и ноль в противном случае. Точка сама с собой не связывается, поэтому диагональ остаётся нулевой.

```
if (i != j && distances[i][j] <= threshold) {
    links[i][j] = 1;    /* точки считаются соседями */
}
```

После этого шага геометрия исчезает: остаётся граф – набор точек (их называют вершинами) и связей между ними (рёбер). Число единиц в строке –

это число соседей точки. В учебном примере у самой плотно окружённой точки оказалось 48 соседей, а всего точек 240.

Порог выбран правильно, если внутри сгустка точки связаны, а между сгустками связей нет. Слишком маленький порог разрывает сгустки на части и даёт много мелких кластеров; слишком большой склеивает всё облако в один кластер. Как подобрать порог не на глаз, сказано ниже.

Связные компоненты

Кластер на языке графа – это связная компонента: множество вершин, между любыми двумя из которых есть путь по рёбрам, и при этом ни одно ребро не ведёт наружу. Найти все компоненты означает найти все кластеры.

Ищут их так: берётся ещё не помеченная точка, ей присваивается очередной номер кластера, и от неё обходом по связям помечаются все достижимые точки. Когда обход заканчивается, компонента найдена целиком; переходим к следующей непомеченной точке и повторяем. Число повторов и есть число кластеров.

Обход можно делать двумя способами, и различаются они только тем, в каком порядке точки ждут своей очереди.

Волновой алгоритм: лесной пожар

Первая точка «поджигается» и получает номер волны 1. Все её непройденные соседи получают номер 2, их непройденные соседи – 3, и так далее: огонь расходится кольцами, пока есть куда идти. Точки, ожидающие обработки, хранятся в очереди – структуре, из которой берут в том же порядке, в каком клали. Именно очередь заставляет алгоритм сначала обработать всё кольцо и только потом перейти к следующему.

```
std::queue<std::size_t> fire;

wave[start] = 1;                      /* поджигаем точку */
fire.push(start);

while (!fire.empty()) {
    std::size_t current = fire.front();

    fire.pop();
    for (std::size_t next = 0; next < n; ++next) {
        if (links[current][next] == 1 && wave[next] == 0) {
            wave[next] = wave[current] + 1;    /* следующее кольцо */
            cluster[next] = number;
            fire.push(next);
        }
    }
}
```

У точек остаётся не только номер кластера, но и номер волны, а это полезная величина: она равна числу шагов по связям от точки поджига, то есть кратчайшему пути внутри кластера. Наибольший номер волны показывает, насколько кластер «растянут»: в учебном примере огню хватило семи волн, чтобы дойти до самой дальней точки.

Проверка на непройденность (`wave[next] == 0`) обязательна – без неё точки, связанные в кольцо, будут бесконечно поджигать друг друга.

Прорубание просеки

Второй способ отличается одной деталью: вместо очереди берётся стек – структура, из которой достают последнее положенное. Тогда обход не расходится кольцами, а уходит по связям как можно дальше от начала, и лишь упёршись, возвращается к отложенным точкам и продолжает оттуда.

```
std::vector<std::size_t> path;    /* стек отложенных точек */

cluster[start] = number;
path.push_back(start);

while (!path.empty()) {
    std::size_t current = path.back();    /* берём последнюю */

    path.pop_back();
    for (std::size_t next = 0; next < n; ++next) {
        if (links[current][next] == 1 && cluster[next] == 0) {
            cluster[next] = number;
            path.push_back(next);
        }
    }
}
```

Результат обязан совпасть с волновым алгоритмом: состав связной компоненты не зависит от того, в каком порядке её обходили. Различаются только промежуточные данные – номеров волн здесь нет. Совпадение двух независимых обходов и служит проверкой: в учебном примере оба нашли 4 кластера размерами 80, 100, 20 и 40 точек.

Стоимость обхода – просмотр всей матрицы связей, то есть снова квадрат числа точек. Если хранить не матрицу, а для каждой точки список её соседей, обход обойдётся числом рёбер вместо квадрата, но матрицу всё равно придётся один раз построить.

Порог по минимальному покрывающему дереву

Подбирать порог вручную неудобно: он зависит от плотности облака и от масштаба задачи. Есть способ получить его из самих данных.

Минимальное покрывающее дерево – набор из n минус одной связи, который соединяет все точки без замкнутых путей и имеет наименьшую возможную суммарную длину. Строится оно алгоритмом Прима: начинается с одной точки, и на каждом шаге к уже связанной части добавляется ближайшая к ней внешняя точка вместе с соответствующей связью.

Смысл такой: внутри сгустка точки густые, и связи между ними короткие; чтобы перебраться в соседний сгусток, дереву приходится один раз сделать длинный шаг. Поэтому если выписать длины всех связей дерева по возрастанию, в этом ряду обнаружится заметный скачок – граница между «внутри сгустка» и «между сгустками». Порогом берут середину самого большого скачка.

```
std::sort(edges.begin(), edges.end());

for (std::size_t i = 1; i < edges.size(); ++i) {
    if (edges[i] - edges[i - 1] > gap) {
        gap = edges[i] - edges[i - 1];
    }
}
```

```

    threshold = (edges[i] + edges[i - 1]) / 2.0f;
  }
}

```

В учебном примере самая короткая связь дерева вышла 0.01, самая длинная 4.11, а предложенный порог – 2.39. С ним оба обхода и нашли четыре кластера, что совпадает с тем, как точки рассыпались на самом деле.

У способа есть предел применимости: он опирается на то, что сгустки действительно разделены пустотой. Если группы соприкасаются, заметного скачка в длинах связей не будет, и любой порог окажется спорным.

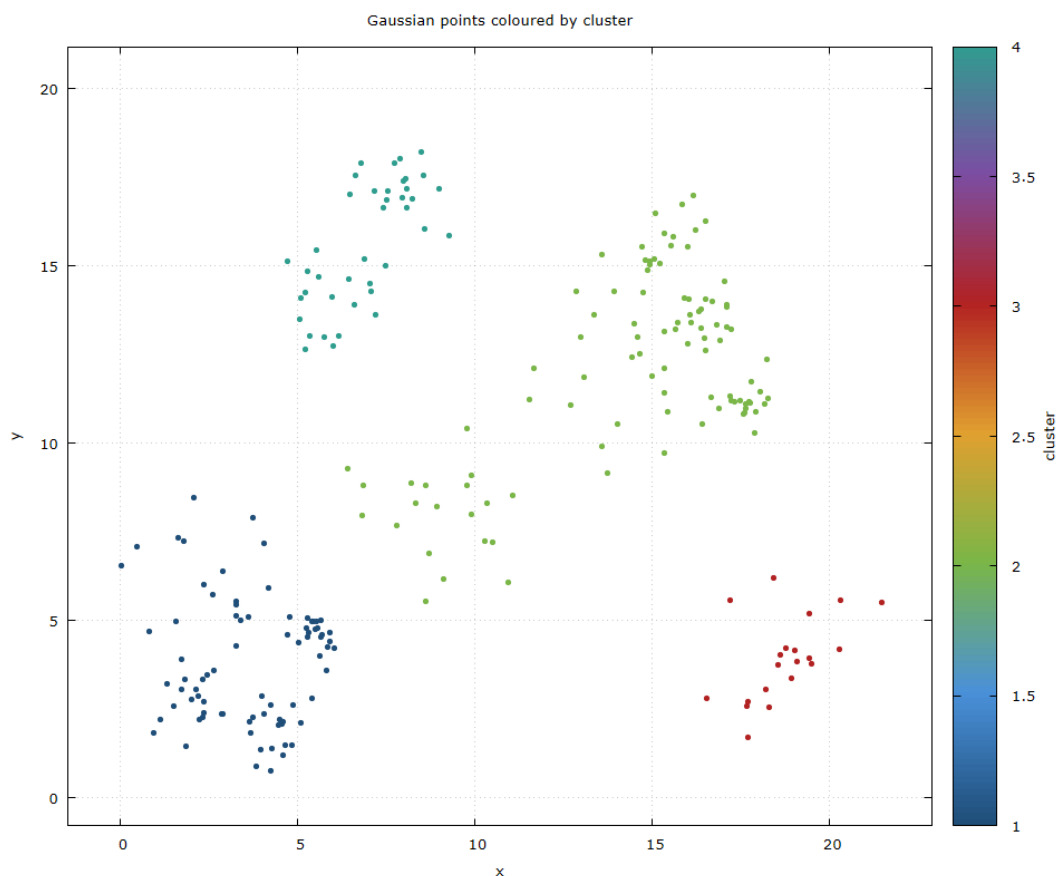


Рисунок 1. Найденные кластеры: цвет точки – номер её связной компоненты

Сетка блоков

Квадратичная цена матрицы расстояний – главное ограничение всей цепочки. Обойти его помогает разбиение плоскости на равные прямоугольные блоки: по координатам точки сразу вычисляется номер блока, в который она попала, и точка запоминает его вместе с координатами.

Дальше используется простое соображение: близкие точки лежат либо в том же блоке, либо в соседних. Поэтому при поиске ближайшей точки сначала просматривается блок искомой точки, затем кольцо соседних блоков, затем следующее кольцо; расширение прекращается, как только уже найденное расстояние оказывается меньше расстояния до границы очередного кольца – за ней ничего ближе быть не может.

В учебном примере при сетке 10 на 10 блоков полный перебор потребовал 240 вычислений расстояния, а поиск по блокам – 26, то есть на 214 меньше при том же ответе. Совпадение ответов здесь принципиально: ускорение не

должно менять результат.

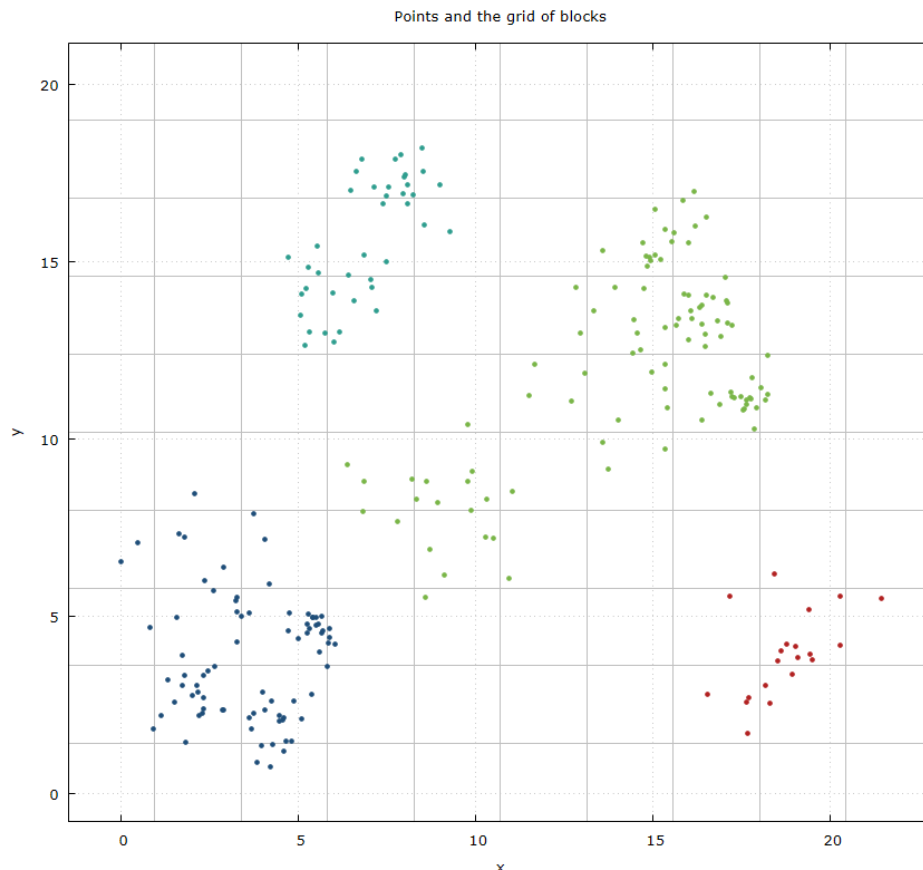


Рисунок 2. Сетка блоков и распределение точек по ней

Тем же приёмом сокращают и построение матрицы связей: раз связи возникают только между точками ближе порога, размер блока берут равным порогу, и тогда для каждой точки достаточно проверить соседние блоки, а не всё облако. Выигрыш зависит от равномерности: если все точки попали в один блок, перебор снова становится полным.

Замечание о признаках и размерности

Всё описанное нигде не опиралось на то, что точки лежат на плоскости. Нужно лишь уметь измерять расстояние между объектами. Если объект описан не двумя числами, а десятком признаков, расстояние считается по тем же правилам в пространстве признаков, и матрица расстояний, порог, двоичная матрица и обходы работают без изменений.

Меняется только геометрическая интуиция: в пространстве многих признаков расстояния между случайными объектами становятся почти одинаковыми, и разница между «близко» и «далеко» размывается – выбрать осмысленный порог становится тем труднее, чем больше признаков.

Стоит отметить и родство с нейронными сетями на радиальных функциях. Такая сеть устроена как набор центров, и каждый центр откликается тем сильнее, чем ближе к нему предъявленный объект; отклик убывает с расстоянием по колоколообразному закону. По сути это тот же кластерный анализ, только вместо жёсткого порога используется плавное убывание, и принадлежность объекта группе оказывается не «да или нет», а числом от нуля до единицы. Центры для такой сети как раз и берут из результатов

кластеризации.

Заключение

Вся цепочка состоит из четырёх шагов, и каждый следующий работает с результатом предыдущего: расстояния, порог, связи, компоненты. Главное решение – порог, и его лучше брать из данных по скачку длин связей покрывающего дерева, а не назначать. Проверка результата тоже встроена в цепочку: два разных обхода должны дать одинаковые кластеры, а ускоренный поиск – тот же ответ, что и полный перебор.

*[координаты точек] --> [матрица расстояний $n \times n$]
[матрица расстояний] -(порог)-> [двоичная матрица связей]
[двоичная матрица] -(волна / просека)-> [номера кластеров]*

Приложение. Памятка по шагам расчёта

Шаг	Что строится	Как	Цена
Расстояния	матрица $n \times n$	попарно, считается верхняя половина	квадрат числа точек
Порог	одно число	скачок длин рёбер покрывающего дерева	сортировка n чисел
Связи	матрица из 0 и 1	единица, если расстояние не больше порога	квадрат числа точек
Соседи	число на точку	сумма единиц в строке	квадрат числа точек
Кластеры волной	номер кластера и волны	очередь, обход кольцами	просмотр матрицы связей
Кластеры просекой	номер кластера	стек, обход вглубь	просмотр матрицы связей
Ускорение	сетка блоков	проверяются блок и его соседи	зависит от равномерности